Capstone Project

Integrity Risk Assessment Agent

Problem statement

Organizations evaluating third-party integrity must synthesize signals from sanctions lists, regulatory filings, ESG databases, financial forecasts, and reputational news, all in real time. Manual workflows are slow, inconsistent, and prone to critical oversight gaps. A monolithic single-agent design compounds these risks: one agent cannot simultaneously handle planning, retrieval, disambiguation, scoring, synthesis, and quality control without becoming a single point of failure.

The result is brittle pipelines that hallucinate under ambiguity, miss conflicts across data sources, and cannot scale to concurrent assessments. For high-stakes compliance use cases, this is not an acceptable baseline. A fundamentally different architectural approach is required.

Why multi-agent?

A stand-alone LLM or monolithic pipeline is insufficient for five reasons:

Real-time grounding: LLMs cannot fetch live sanctions updates, regulatory filings, or breaking news. External retrieval tools are mandatory.

Hallucination risk: Factual risk claims require verifiable evidence with provenance. LLMs generate plausible-sounding but ungrounded answers.

Computational tasks: Financial distress forecasting, anomaly detection, and structured risk scoring require external models, not text generation.

Persistent memory: Historical risk profiles, calibration data, and trend detection require durable storage that persists across sessions.

Workflow complexity: Coordinating multi-source retrieval, conflict resolution, forecasting, and synthesis cannot be collapsed into a single prompt.

A multi-agent system distributes these responsibilities across specialized agents, each with a focused capability, coordinated by a central orchestrator. This separation enables parallel execution, independent failure handling, and clear auditability, properties that a monolithic approach cannot achieve at the required reliability level.

Architecture overview

Agent	Role	Key Responsibilities
Orchestrator Agent	Central coordinator; manages the ReAct reasoning loop (Think → Act → Observe → Revise → Conclude)	Interprets user intent; decomposes the question into risk dimensions (financial, sanctions, ESG, regulatory, reputational, operational); generates and adapts the assessment plan; routes tasks to specialist agents; collects observations; synthesizes the final risk conclusion with confidence levels and recommends next steps.
Retrieval Agent	Real-time data gatherer	Queries sanctions APIs (OFAC, EU, UN), regulatory databases (SEC, FCA), enforcement databases, news feeds, and ESG incident databases; performs entity resolution to match name variants; returns structured evidence items with source, timestamp, and confidence metadata.
Analysis & Forecasting Agent	Quantitative risk modeler	Ingests structured market data; runs time-series forecasting models for financial distress and liquidity risk; performs anomaly detection; produces numerical risk scores with uncertainty bounds; delegates computation outside LLM capabilities to external tools.
Conflict Resolution Agent (ToT Module)	Hypothesis evaluator for ambiguous or conflicting signals	Activates when signals contradict (e.g., OFAC API says "not sanctioned" but Reuters reports a sanction); generates parallel candidate hypotheses (Tree-of-Thought, beam width 3–5, depth cap 5–6); scores each branch on consistency, temporal coherence, source authority, and cold-start conservatism; selects the winning update plan; preserves runner-up hypotheses as low-confidence alternatives for auditability; outputs a structured conflict resolution note with confidence level.
Memory Manager Agent	Durable state and calibration steward	Maintains short-term working memory (current question, active plan, retrieved evidence, unresolved conflicts) cleared after each session; manages long-term vector database storing summaries of past assessments, historical sanctions, calibration outcomes (false positive/negative rates, source reliability scores), and watchlist rules; updates calibration thresholds after each assessment to create a self-correcting retrieval-reasoning loop; mitigates semantic drift via entity resolution, metadata filtering, and LLM-based relevance scoring.

Agent Count Rationale

The system uses five agents, a number arrived at by mapping distinct, non-overlapping capability requirements to dedicated roles. Task complexity analysis identified five genuinely separable concerns:

1. multi-step reasoning and plan management, which require a dedicated orchestrator that cannot also perform retrieval without creating coupling between planning and execution;
2. real-time data access across heterogeneous external APIs, which requires its own tool-calling context isolated from reasoning state;
3. quantitative computation, forecasting and anomaly detection, which requires external model invocation that would bloat the retrieval agent if co-located;
4. conflict resolution under ambiguity, which requires a branching search strategy (ToT) fundamentally incompatible with the linear execution mode of the retrieval and analysis agents; and
5. durable state and calibration management, which requires write access to the vector database and must be isolated from agents that only read from it to prevent race conditions and corrupted calibration signals.

A four-agent design was considered: merging the Analysis & Forecasting Agent with the Retrieval Agent. This was rejected because quantitative model execution and real-time API retrieval have different latency profiles, failure modes, and retry strategies, coupling them would make the retrieval path slower and harder to fail-recover independently. A six-agent design was also considered, splitting the Orchestrator into a Planner and a Synthesizer. This was rejected as offering diminishing returns: the Orchestrator's planning and synthesis steps share reasoning context so tightly that separating them would require constant state synchronization with no isolation benefit. Five agents represent the minimum necessary specialization to handle all capability requirements while avoiding coordination overhead that would outweigh the separation benefit.

Coordination and Communication Strategy

Coordination Pattern: Hybrid Sequential-Parallel with Iterative Feedback

The system uses a hybrid coordination strategy that combines elements of sequential, parallel, and iterative graph-based workflows — chosen because no single pure pattern is sufficient for the task.

The outer loop is iterative: the Orchestrator's ReAct cycle (Think → Act → Observe → Revise → Conclude) repeats until sufficient evidence is assembled or a confidence threshold is met.

Within each iteration, execution is a directed graph: the Orchestrator dispatches tasks to the Retrieval Agent and Memory Manager Agent in parallel (fan-out), waits for their results (join), then conditionally branches, activating the Conflict Resolution Agent only when contradictions are detected, and the Analysis Agent only when quantitative signals are required. The final synthesis step is sequential: it executes only after all upstream results are resolved.

This hybrid pattern was chosen over alternatives for specific reasons. A purely sequential (assembly-line) workflow would require retrieval to complete before analysis begins, wasting latency on tasks that are independent. A purely parallel workflow would lack the conditional logic needed to invoke the ToT module only when conflicts exist, running expensive hypothesis search on every query. A fully graph-based workflow without a central orchestrator would require peer-to-peer negotiation between agents, increasing coordination complexity and making the workflow harder to trace and audit. The hub-and-spoke hybrid gives the Orchestrator clear visibility into workflow state while enabling parallel execution where tasks are independent.

Shared state bus: All agents read and write to a shared Model Context Protocol (MCP) store that holds the current branch state, active beam candidates (for ToT), retrieved evidence items, and per-branch scoring metadata. MCP is the authoritative state layer; no agent holds private state beyond its current task.

Orchestration framework: CrewAI manages agent roles, task delegation, and lifecycle. LangChain provides prompt templates, tool wrappers, and evaluation chains for heuristic scoring. MCP maintains branch and global state.

Execution flow:

- User submits a natural-language risk query.
- Orchestrator interprets intent, decomposes into risk dimensions, and generates an initial plan.
- Orchestrator fans out parallel retrieval tasks to the Retrieval Agent across sanctions, regulatory, news, and ESG sources simultaneously, while the Memory Manager Agent retrieves top-5 historical semantic matches and calibration facts from the vector DB.
- Retrieval results are observed and normalized by the Orchestrator. If conflicts are detected, the Conflict Resolution (ToT) Agent is invoked.
- If quantitative signals are needed, the Analysis & Forecasting Agent runs concurrently with conflict resolution.

- The ToT module returns a resolved update plan; the Memory Manager writes stable facts to the vector DB post-assessment.
- Orchestrator synthesizes all evidence into a final assessment with risk rating, confidence level, and next steps.

Retrieval timing: Retrieval occurs at the start of reasoning, after each major observation, during conflict resolution, and before final synthesis, ensuring retrieval actively shapes reasoning rather than serving as a one-time lookup.

Communication Modalities

Agents do not communicate peer-to-peer. All inter-agent information exchange flows through the Orchestrator or the shared MCP state bus, using three distinct communication patterns depending on the relationship:

One-way (downstream instruction): The Orchestrator issues task instructions to the Retrieval Agent, Analysis Agent, and Memory Manager Agent at the start of each fan-out. These agents return structured results but do not initiate communication or send requests back. This pattern is used where the task is fully specified and no negotiation is needed, keeping latency low.

Two-way (validation and feedback): The Orchestrator and the Conflict Resolution (ToT) Agent engage in iterative two-way communication. The Orchestrator submits a conflict description and candidate hypotheses; the ToT module returns scored branches; the Orchestrator may issue follow-up retrieval instructions that feed back into the ToT module's next evaluation round. This bidirectional loop is necessary for conflicts where the resolution depends on evidence retrieved in response to the first hypothesis evaluation.

Brainstorming (multi-path exploration): Within the ToT module itself, the thought generator produces multiple competing hypotheses simultaneously (beam width 3–5) and the critic agent scores all branches in parallel before the controller selects a winner. This is the system's only true brainstorming pattern, deliberately contained within the Conflict Resolution Agent so that exploratory multi-path reasoning does not propagate latency into the main workflow.

The Memory Manager Agent operates as a passive shared state participant: it writes calibration updates and historical summaries post-assessment and responds to semantic retrieval requests but never initiates communication.

Design Rationale and Trade-offs

Specialization vs. Coordination Overhead. Distributing work across five agents introduces inter-agent communication and orchestration cost. This overhead is justified because each agent can fail, retry, and be upgraded independently. A single monolithic agent that fails mid-workflow loses all intermediate work; in the hub-and-spoke design, only the failing spoke is retried. The MCP shared state ensures that partial results survive individual agent failures.

Tree-of-Thought Accuracy vs. Latency. The ToT conflict resolution module explores multiple hypotheses in parallel (beam width 3–5), which increases latency relative to a linear chain-of-thought approach. This cost is accepted because linear CoT prematurely commits to a single interpretation and propagates errors into the memory store, a critical failure mode in compliance contexts where an overwritten historical fact or an incorrectly canonicalized entity can produce materially wrong risk conclusions. Strict beam width caps (3–5) and depth caps (5–6) bound compute to acceptable enterprise latency budgets.

Cold-Start Conservatism vs. Decisiveness. When the vector DB has sparse or no historical memory for a newly encountered entity, the system defaults to conservative, non-destructive memory writes and preserves alternative hypotheses rather than committing to a single interpretation. This reduces false confidence for new entities at the cost of less decisive early assessments. The trade-off is deliberately asymmetric: a wrong confident answer in a compliance context carries greater risk than a calibrated "medium confidence, pending corroboration" result.

Hybrid Retrieval Complexity vs. Reliability. Combining real-time API retrieval with semantic vector DB retrieval adds architectural complexity (two retrieval paths, entity resolution, metadata filtering). This complexity is required because neither path alone is sufficient: real-time retrieval provides recency but no historical pattern awareness; semantic retrieval provides historical context but cannot detect events from last week. Together, they enable the calibrated, conflict-aware conclusions that distinguish the system from a simple API wrapper or a prompt-only LLM.

Scalability and Effective Problem Solving

The architecture is designed to scale along three independent axes without requiring a fundamental redesign.

Horizontal task scaling. Because retrieval tasks are dispatched as parallel fan-out instructions across independent data sources (sanctions, regulatory, news, ESG), adding a new data source requires only registering a new tool with the Retrieval Agent, no changes

to the Orchestrator, Memory Manager, or Conflict Resolution Agent are needed. The retrieval layer scales by adding tool connectors, not by modifying reasoning logic.

Vertical reasoning depth. The ToT module's beam width and depth caps are configuration parameters. For higher-stakes assessments requiring deeper hypothesis exploration, beam width can be increased from 3 to 5 or the depth cap extended from 5 to 6 steps without changing the surrounding workflow. For time-sensitive queries where latency must be minimized, beam width can be reduced to 2 with a single-step fallback. This allows reasoning depth to scale with risk tolerance and time budget on a per-query basis.

Memory and calibration scaling. The vector database grows continuously as assessments accumulate. Retrieval quality is maintained through metadata filtering, entity resolution, and LLM-based relevance scoring, which prevent the top-5 semantic matches from degrading as the corpus grows. Calibration feedback accumulates per entity and per source, meaning the system becomes more accurate, not noisier, as it processes more assessments.

Effective problem solving is supported by three structural properties of the design. First, the ReAct loop ensures the agent does not commit to a conclusion before evidence warrants it, the Revise step explicitly re-plans when observations are contradictory or incomplete. Second, the calibration feedback loop creates a self-correcting system: source reliability scores, false positive rates, and conflict-resolution thresholds are updated after each assessment, so errors from one assessment reduce the probability of the same error recurring. Third, auditability is built into the output structure, every conclusion includes its evidence chain, source provenance, confidence level, and preserved alternative hypotheses, enabling human reviewers to validate, override, or escalate conclusions without having to reconstruct the reasoning from scratch.